LÖSUNGEN · LEHREREXEMPLAR

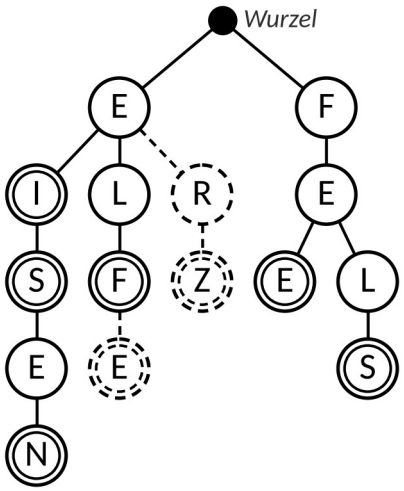
W5·3 – Transfer: Unbekannte Bäume erschließen – Lösungen

Zum Arbeitsblatt `AQ_W5_AB3_Transfer_unbekannte_Baeume.docx` · alle Runenbäume, Wörter, Rückgaben, Knotenzahlen und Rekursionstabellen mit Java (javac 21) und einem exakten Nachbau von BinTree erzeugt; die abgedruckten Methoden (enthaelt, anzahlWoerter, raetsel, istPraefix, enthaelt mit Abbruch) wurden gegen eine unabhängige Wortliste geprüft · Klammerform: Buchstabe, * = Wortende, links = erstes Kind (K), rechts = nächster Geschwister (G) · Anhang: Testumgebung „Runenbuch“ für den Papier-Compiler

Teil 1 – Die Strategie am Runenbaum

Aufgabe 1 – Lesen und von Hand nachvollziehen (8 BE)

a) EIS, EISEN, ELF, FEE, FELS (1 BE). **b)** EIS endet bei S; EISEN führt über dasselbe S weiter (S hat das Kind E). Das Ende von EIS ist also ein innerer Knoten. Ohne Markierung könnte man nicht unterscheiden, ob ein Weg ein gespeichertes Wort ist oder nur der Anfang eines längeren: Der Weg E–I–S–E existiert, EISE ist aber kein Wort (2 BE).



Präfixbaum nach ELFE, EI und ERZ (gestrichelt: neue Knoten; I ist jetzt markiert)

c) ELFE: **+1** Knoten (E unter F, markiert) · EI: **+0** – der Weg E–I gibt es schon, I wird nur markiert · ERZ: **+2** (R als neues drittes Kind von E – alphabetisch nach L –, darunter Z markiert). Knoten ohne Wurzel: 12 → 13 → 13 → 15; gespeichert: EI, EIS, EISEN, ELF, ELFE, ERZ, FEE, FELS (3 BE).

d) Von der Wurzel aus für jeden Buchstaben des Worts unter den Kindern des aktuellen Knotens das Kind mit diesem Buchstaben suchen und hinabsteigen. Gibt es nach dem letzten Buchstaben einen markierten Knoten, ist das Wort gespeichert. „Nein“, wenn (1) unterwegs ein Buchstabe fehlt (z. B. FEUER: unter F–E gibt es kein U) oder (2) der Weg vollständig existiert, der letzte Knoten aber nicht markiert ist (z. B. EISE; im Ausgangsbaum auch EI) (2 BE).

Aufgabe 2 – Was gilt immer? (6 BE)

a) Drei aus (je 1 BE): (1) Die Kinder eines Knotens enthalten **verschiedene** Buchstaben und sind **alphabetisch** geordnet. (2) **Jedes Blatt ist markiert** – Knoten entstehen nur beim Einfügen eines Worts, und dessen letzter Knoten wird markiert. (3) Zu jedem gespeicherten Wort gibt es **genau einen** Weg von der Wurzel zu einem markierten Knoten; jeder Wortanfang eines gespeicherten Worts ist ein Weg im Baum. (4) Die Tiefe eines Knotens ist die Länge seines Wortanfangs.

b) Wegen (1) bestimmt jedes Wort genau einen Weg und damit genau einen Endknoten, und verschiedene Wörter enden in verschiedenen Knoten. Jeder markierte Knoten gehört umgekehrt zu genau einem Wort (den Buchstaben auf seinem Weg). Also: Anzahl Wörter = Anzahl markierter Knoten (1 BE).

c) Die Knoten entsprechen genau den Wortanfängen der gespeicherten Wörter, die Reihenfolge der Kinder ist durch das Alphabet festgelegt, markiert sind genau die Wörter. Die Gestalt hängt also nur von der

Menge der Wörter ab, nicht von der Reihenfolge (per Programm mit mehreren Reihenfolgen geprüft). Beim Suchbaum entscheidet dagegen der zuerst eingefügte Wert über die Wurzel – andere Reihenfolge, andere Gestalt und andere Höhe (AB 2, Aufgabe 3) (2 BE).

Aufgabe 3 – Auf BinTree abbilden (10 BE)

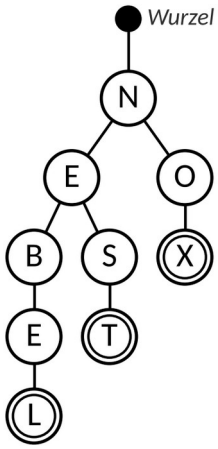
Teilbaum	getZeichen()	istEnde()
`t.getLeft()`	I	falsch
`t.getRight()`	F	falsch
`t.getLeft().getLeft()`	S	wahr
`t.getLeft().getRight()`	L	falsch
`t.getRight().getLeft().getLeft().getRight()`	L	falsch

a) 3 BE (fünf Zeilen mit je zwei Werten). Typischer Fehler: getRight als „rechtes Kind“ lesen – es ist das nächste **Geschwister** (F ist Geschwister von E, nicht Kind).

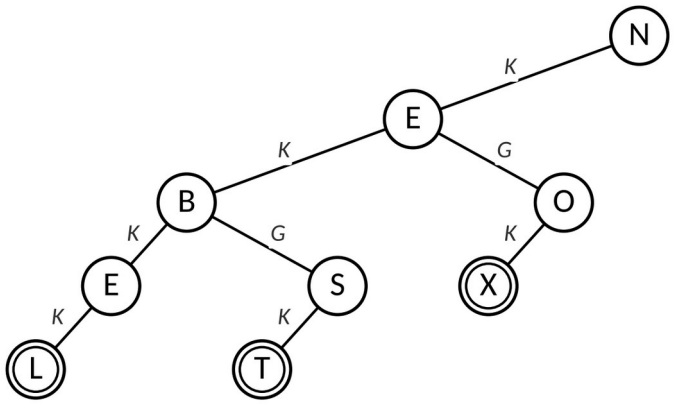
Baumart	Inhalt eines Knotens	links / rechts	Invariante (gilt immer)
Runenbaum	Rune: ein Buchstabe und die Markierung Wortende (Cast `(Rune) b.getItem()`)	erstes Kind (nächster Buchstabe des Worts) / nächster Geschwister (anderer Buchstabe an derselben Stelle)	jede rechte Kette alphabetisch ohne doppelte Buchstaben; Knoten mit leerem linken Teilbaum sind Wortenden

b) 2 BE.

Mehrwegbaum



Binärbaum (K = erstes Kind, G = nächster Geschwister)



c) Binärbaum `N(E(B(E(L*,-),S(T*,-)),O(X*,-)),-)` – 9 Knoten, Höhe 5. NEBEL und NEST teilen sich N–E; unter E folgen die Geschwister B und S (alphabetisch); O ist Geschwister von E. Häufiger Fehler: X als Kind von T statt von O (3 BE).

d) Höhe **6**, längstes Wort **5** Buchstaben (EISEN). Der Weg im Binärbaum enthält auch **G-Kanten** zu Geschwistern; sie verlängern den Weg, aber nicht das Wort. Beispiel: E –G→ F –K→ E –K→ E –G→ L –K→ S hat 6 Knoten, liest aber nur F, E, L, S (FELS). Die Höhe ist also höchstens (längste Wortlänge) + (Anzahl der Geschwister-Schritte) (2 BE).

Aufgabe 4 – Fremden Code lesen (8 BE)

Rekursionstiefe	Knoten	wort	Fall	Rückgabe
0	E	"FELS"	E ≠ F → rechts	wahr
1	F	"FELS"	F = F → links	wahr
2	E	"ELS"	E = E → links	wahr
3	E*	"LS"	E ≠ L → rechts	wahr

Rekursionstiefe	Knoten	wort	Fall	Rückgabe
4	L	"LS"	L = L → links	wahr
5	S*	"S"	letzter Buchstabe → istEnde()	wahr

a) 4 BE: sechs Aufrufe, Tiefe 0 bis 5; das Wort wird nur bei einem Schritt nach **links** gekürzt.

wort	"EI"	"EISE"	"FELSEN"	"E"
Rückgabe	falsch	falsch	falsch	falsch
untersuchte Knoten	2	4	6	1

b) 2 BE. EI: Weg vorhanden, I nicht markiert. EISE: letzter Knoten E nicht markiert. FELSEN: nach dem markierten S ist der linke Teilbaum leer (Buchstabe E fehlt). E: E ist nicht markiert.

c) Zeile 7 (rechter Teilbaum, wort unverändert): Im Präfixbaum wird **derselbe** Buchstabe unter den **anderen Kindern desselben Elternknotens** gesucht – gleiche Ebene, nächstes Geschwister. Zeile 13 (linker Teilbaum, wort ohne ersten Buchstaben): Der Buchstabe ist gefunden; es geht **eine Ebene tiefer** zu den Kindern, gesucht wird der nächste Buchstabe (2 BE).

Aufgabe 5 – Implementieren (4 BE + Papier-Compiler)

```
public static int anzahlWoerter(BinTree b) {
    if (b.isEmpty()) {
        return 0;
    }
    Rune r = (Rune) b.getItem();
    int anzahl = anzahlWoerter(b.getLeft()) + anzahlWoerter(b.getRight());
    if (r.istEnde()) {
        anzahl = anzahl + 1;                // hier endet ein Wort
    }
    return anzahl;
}
```

a) 4 BE: Signatur · Basisfall leer → 0 · Cast `(Rune)` · **beide** Teilbäume rekursiv (Kinder **und** Geschwister) · +1 nur bei istEnde(). Kontrollwerte: M2a 5, nach ELFE/EI/ERZ 8, Abbildung M4 5, leer 0. Gleichwertig: Blätter zählen ist **falsch** (EIS ist kein Blatt); Knoten mit leerem linken Teilbaum zählen ebenfalls falsch (EI, EIS werden nicht gezählt).

b) Testumgebung im Anhang. Mit der Musterlösung ist `papier_anzahlWoerter` grün; diese typischen Fehler macht der Test rot: alle Knoten gezählt, Geschwister vergessen, „linker Teilbaum leer“ statt istEnde(), Basisfall vergessen (Laufzeitfehler).

Teil 2 – Mini-Abi: Das Zauberbuch der Waldhüterin (18 BE)

Teilaufgabe	Operator / Inhalt	BE	AFB I	AFB II	AFB III
a)	angeben: gespeicherte Wörter	2	2		
b)	darstellen: Binärbaum nach Einfügen	3	2	1	
c)	bestimmen: Rückgabe und untersuchte Knoten	2	1	1	
d)	analysieren, beschreiben, erläutern: raetsel	4		3	1
e)	implementieren: istPraefix	4		3	1
f)	beurteilen: Abbruch bei kleinerem Buchstaben	3		1	2
Summe		18	5 (28 %)	9 (50 %)	4 (22 %)

a) MOLCH, MOND, MOOR, MOOS, NEBEL (2 BE). Die Preorder des Binärbaums liefert die Wörter bereits alphabetisch (Invariante (1)).

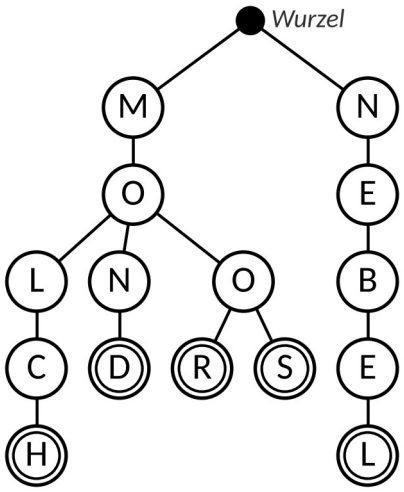
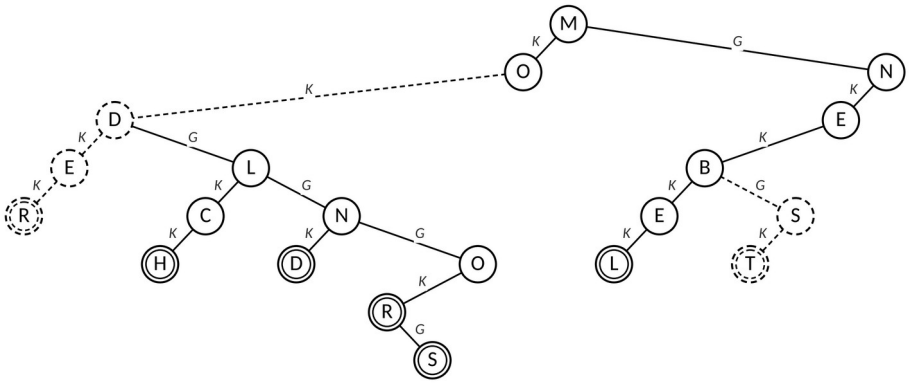


Abbildung M4 als Präfixbaum (Hilfe zum Ablesen)



Nach MODER und NEST (gestrichelt: neue Knoten)

b) MODER: D ist alphabetisch kleiner als L und wird **erstes Kind** von O – D wird linker Teilbaum von O, L rückt als Geschwister nach rechts; darunter E und das markierte R (+3 Knoten, 18 insgesamt). NEST: unter N–E ist S Geschwister von B (+2 Knoten). Ergebnis `M(O(D(E(R*,-),L(C(H*,-),N(D*,O(R*(-,S*),-)))),-),N(E(B(E(L*,-),S(T*,-)),-),-))`, 20 Knoten (3 BE: D vor L einsortiert 1 BE, MODER-Kette 1 BE, NEST 1 BE).

c) MOOS: **wahr**, 7 Knoten (M, O, L, N, O, R, S) · MOO: **falsch**, 5 Knoten (das zweite O ist nicht markiert) · MOHN: **falsch**, 5 Knoten (M, O, L, N, O – kein H unter den Kindern von O) (2 BE).

d) `raetsel(t)` = **5**. raetsel liefert die **Länge des längsten gespeicherten Worts** (die Tiefe des Präfixbaums): Der linke Teilbaum setzt das Wort um einen Buchstaben fort (+1), der rechte Teilbaum enthält **Alternativen an derselben Stelle** – ein Geschwister verlängert das Wort nicht, deshalb kein +1. Das Maximum aus beiden ist die längste Fortsetzung ab hier. Weil jedes Blatt markiert ist, endet der längste Weg in einem Wort. Zum Vergleich: Die Höhe des Binärbaums ist 7 (4 BE: Wert 1, Bedeutung 2, Begründung +1 nur links 1).

```
public static boolean istPraefix(BinTree b, String p) {
    if (b.isEmpty()) {
        return false;                       // Buchstabe fehlt
    }
    Rune r = (Rune) b.getItem();
    if (r.getZeichen() != p.charAt(0)) {
        return istPraefix(b.getRight(), p);  // bei den Geschwistern suchen
    }
    if (p.length() == 1) {
        return true;                        // Weg vorhanden - Wortende egal
    }
    String rest = p.substring(1, p.length());
    return istPraefix(b.getLeft(), rest);
}
```

e) 4 BE: Signatur · Basisfall leer → falsch · Geschwister-Suche bei anderem Buchstaben · Rückgabe **wahr** beim letzten Buchstaben (**ohne** istEnde) · Abstieg mit `p.substring(1, p.length())`. Kontrollwerte Abbildung M4: MO wahr, MOO wahr, MOOSE falsch, NE wahr, N wahr, MOL wahr, MI falsch, O falsch, MOLCH wahr, NEBELN falsch; leerer Baum falsch. Häufiger Fehler: `return r.istEnde()` übernommen (MOO würde falsch) – `papier_istPraefix` wird dann rot.

```
public static boolean enthaelt(BinTree b, String wort) {
    if (b.isEmpty()) {
        return false;
    }
    Rune r = (Rune) b.getItem();
    char c = wort.charAt(0);
    if (c < r.getZeichen()) {
        return false;                       // NEU: c kann nicht mehr folgen
    }
    if (c > r.getZeichen()) {
        return enthaelt(b.getRight(), wort);
    }
    if (wort.length() == 1) {
        return r.istEnde();
    }
    String rest = wort.substring(1, wort.length());
    return enthaelt(b.getLeft(), rest);
}
```

wort	MOOS	MOO	NEBEL	MAUS	MOHN	LUCHS
Rückgabe	wahr	falsch	wahr	falsch	falsch	falsch
untersuchte Knoten vorher	7	5	6	2	5	2
… mit Abbruch	7	5	6	2	3	1

f) 3 BE. **Korrektheit:** Die Geschwister sind alphabetisch aufsteigend geordnet (Invariante). Ist der gesuchte Buchstabe kleiner als der aktuelle, kann er weiter rechts nicht mehr vorkommen – der Abbruch ändert kein Ergebnis (für alle Testwörter geprüft). **Aufwand:** Gespart wird nur bei Wörtern, die **nicht** gespeichert sind und deren Buchstabe „zwischen“ zwei Geschwister fällt (MOHN, LUCHS); bei gespeicherten Wörtern muss man ohnehin bis zum Buchstaben laufen. Im schlechtesten Fall bleibt alles gleich: Ein Runenbaum mit allen 17.576 Wörtern von AAA bis ZZZ braucht mit und ohne Abbruch höchstens 78 / 78 untersuchte Knoten (26 Geschwister je Stelle × 3 Stellen). **Fazit:** korrekt und sinnvoll, aber nur eine kleine Verbesserung – die Größenordnung (höchstens 26 · Wortlänge) bleibt.

Für Schnelle

woerter(b, praefix): leer → ""; sonst w ← praefix + Zeichen; Ergebnis = (w + " ", falls istEnde) + woerter(links, w) + woerter(rechts, praefix). Das ist eine **Preorder** – sie liefert die Wörter alphabetisch (z. B. M2a: `EIS EISEN ELF FEE FELS`). AAA bis ZZZ: 17.576 Wörter, enthaelt untersucht höchstens 78 Knoten. Ein ausgeglichener Suchbaum mit 17.576 Wörtern hat die Höhe 15 – nur 15 Vergleiche, aber jeder ist ein Zeichenketten-Vergleich (compareTo) über bis zu 3 Zeichen. Der Präfixbaum vergleicht immer nur einzelne Zeichen und kann zusätzlich Wortanfänge beantworten (istPraefix).

Anhang – Testumgebung „Runenbuch“ für den Papier-Compiler

Als eigene Datei in einen Workspace mit aktivierter Bibliothek „Niedersachsen“ kopieren (Text aus diesem Kasten) und an die Lernenden ausgeben. Die Lernenden tippen ihre Methoden in den Abschnitt DEIN TEIL ein. Geprüft unter Java 21 mit dem BinTree-Nachbau:

- Gerüst (unverändert): 0/2 Tests grün
- mit der Musterlösung (anzahlWoerter und istPraefix wie abgedruckt): 2/2 Tests grün
- typische Fehler (Mutanten): 8 von 8 erkannt – alle Knoten gezählt; Geschwister vergessen; „linker Teilbaum leer“ statt istEnde(); Basisfall vergessen; istPraefix mit return r.istEnde(); istPraefix ohne Geschwister-Suche; istPraefix: leerer Baum liefert wahr; istPraefix: getLeft statt getRight

```
// ======================================================================
//  RUNENBUCH — Testumgebung fuer den Papier-Compiler (AB 3)
//  Informatik LK · Online-IDE · Algo Quest · Welt 5
//  Bibliothek "Niedersachsen" im Workspace aktivieren!
//  Tippe deine handschriftlichen Methoden UNVERAENDERT in DEIN TEIL ein
//  und starte im Reiter "Testrunner" die Klasse RunenbuchTest.
// ======================================================================

// ------------------- GERUEST — hier nichts veraendern -------------------
class Rune {
    private char zeichen;
    private boolean ende;

    public Rune(char zeichen, boolean ende) {
        this.zeichen = zeichen;
        this.ende = ende;
    }

    public char getZeichen() {
        return zeichen;
    }

    public boolean istEnde() {
        return ende;
    }

    public void setEnde(boolean e) {
        ende = e;
    }
}

class Runenbaum {

    // baue(woerter: Reihung von Zeichenkette): BinTree  — neuer Runenbaum
    public static BinTree baue(String[] woerter) {
        BinTree b = new BinTree();
        for (int i = 0; i < woerter.length; i++) {
            einfuegen(b, woerter[i]);
        }
        return b;
    }

    // einfuegen(b: BinTree vom Inhaltstyp Rune, wort: Zeichenkette): nichts
    // links = erstes Kind, rechts = naechster Geschwister (alphabetisch)
    public static void einfuegen(BinTree b, String wort) {
        char c = wort.charAt(0);
        String rest = wort.substring(1, wort.length());
        if (b.isEmpty()) {
            b.setItem(new Rune(c, rest.length() == 0));
        } else {
            Rune r = (Rune) b.getItem();
            if (c > r.getZeichen()) {
                einfuegen(b.getRight(), wort);
                return;
            }
            if (c < r.getZeichen()) {
                BinTree alt = new BinTree(r);
                alt.setLeft(b.getLeft());
                alt.setRight(b.getRight());
```

```
                b.setItem(new Rune(c, rest.length() == 0));
                b.setLeft(new BinTree());
                b.setRight(alt);
            } else if (rest.length() == 0) {
                r.setEnde(true);
            }
        }
        if (rest.length() > 0) {
            einfuegen(b.getLeft(), rest);
        }
    }

    // ------------------- DEIN TEIL – hier arbeitest du! -------------------

    // TODO AB 3, Aufgabe 5: anzahlWoerter(b: BinTree vom Inhaltstyp Rune): Ganzzahl
    public static int anzahlWoerter(BinTree b) {
        return -1;
    }

    // TODO AB 3, Mini-Abi e): istPraefix(b: BinTree vom Inhaltstyp Rune, p: Zeichenkette):
Wahrheitswert
    public static boolean istPraefix(BinTree b, String p) {
        return false;
    }
}

// ------------------- TESTS – hier nichts veraendern! -------------------
class RunenbuchTest {

    @Test
    public void papier_anzahlWoerter() {
        try {
            String[] w1 = {"EIS", "EISEN", "ELF", "FEE", "FELS"};
            BinTree t = Runenbaum.baue(w1);
            assertEquals(5, Runenbaum.anzahlWoerter(t), "Runenbaum 1 (EIS, EISEN, ELF, FEE, FELS): 5
Woerter");
            Runenbaum.einfuegen(t, "EI");
            assertEquals(6, Runenbaum.anzahlWoerter(t), "nach einfuegen EI (kein neuer Knoten, nur I
markiert): 6 Woerter");
            Runenbaum.einfuegen(t, "ELFE");
            Runenbaum.einfuegen(t, "ERZ");
            assertEquals(8, Runenbaum.anzahlWoerter(t), "nach ELFE und ERZ: 8 Woerter");
            String[] w3 = {"MOOS", "NEBEL", "MOND", "MOLCH", "MOOR"};
            assertEquals(5, Runenbaum.anzahlWoerter(Runenbaum.baue(w3)), "Runenbaum der Waldhueterin: 5
Woerter");
            String[] w4 = {"A"};
            assertEquals(1, Runenbaum.anzahlWoerter(Runenbaum.baue(w4)), "ein Wort aus einem
Buchstaben: 1");
            assertEquals(0, Runenbaum.anzahlWoerter(new BinTree()), "leerer Runenbaum: 0");
        } catch (RuntimeException e) {
            fail("LAUFZEITFEHLER: " + e.getMessage() + " - Basisfall if (b.isEmpty()) vergessen?");
        }
    }

    @Test
    public void papier_istPraefix() {
        try {
            String[] w3 = {"MOOS", "NEBEL", "MOND", "MOLCH", "MOOR"};
            BinTree t = Runenbaum.baue(w3);
```

```
            assertTrue(Runenbaum.istPraefix(t, "MOO"), "MOO ist Anfang von MOOS und MOOR (aber selbst
kein Wort)");
            assertTrue(Runenbaum.istPraefix(t, "NE"), "NE ist Anfang von NEBEL - dazu muss die Suche
bei den Geschwistern von M weitergehen");
            assertTrue(Runenbaum.istPraefix(t, "MOLCH"), "ein ganzes Wort ist auch ein Anfang");
            assertTrue(Runenbaum.istPraefix(t, "N"), "N");
            assertFalse(Runenbaum.istPraefix(t, "MOOSE"), "MOOSE: nach S geht es nicht weiter");
            assertFalse(Runenbaum.istPraefix(t, "MI"), "MI: kein Wort beginnt mit MI");
            assertFalse(Runenbaum.istPraefix(t, "O"), "O: kein Wort beginnt mit O");
            assertFalse(Runenbaum.istPraefix(new BinTree(), "A"), "leerer Runenbaum: false");
        } catch (RuntimeException e) {
            fail("LAUFZEITFEHLER: " + e.getMessage() + " - Basisfall if (b.isEmpty()) vergessen oder
charAt/substring falsch?");
        }
    }
}
```